

Rapport de conduite de projet — AI Engineer

Projet : Coach Vocal FR — terminal vocal embarqué + cerveau IA auto-hébergé

Auteur : Yang FEI

Contexte pédagogique : OpenClassrooms — parcours AI Engineer, projet 15 (portfolio)

Dépôt : <https://github.com/Lesliam/openclassrooms-project15>

Version : structure conforme au template officiel « Rapport de conduite de projet AIE », état technique arrêté au 24/07/2026

1. Contexte et analyse des besoins

1.1 Présentation (organisation et / ou contexte)

Ce projet s'inscrit dans un scénario professionnel simulé : FalDatacorps, société de conseil en données et en intelligence artificielle, recrute des profils d'AI Engineer. Ma manager, Charlotte, me demande de constituer un portfolio démontrant mes compétences par la preuve, articulé autour d'un projet technique personnel mené de bout en bout — du besoin utilisateur jusqu'à la mise en service et à la mesure des performances.

Le projet retenu est un **coach vocal de français** : un terminal autonome posé sur le bureau, capable d'écouter, de corriger et de relancer un apprenant. Ce choix place le projet à la jonction de mes deux carrières — neuf années d'ingénierie logicielle embarquée (piles BLE automobiles, contraintes mémoire et temps réel) pour la culture du bord, le parcours AI Engineer pour le cycle de vie des modèles.

Je pars d'une base technique réelle — station de travail auto-administrée (Linux, RTX 5080 16 Go) et chaîne vocale auto-hébergée déjà en service à la maison — et de trois contraintes assumées : fonctionnement **100 % local**, budget mémoire GPU de 16 Go partagé avec les services vocaux du foyer, et un sprint de 4 semaines ponctué de trois points de mentorat.

1.2 Collecte et analyse du besoin métier

Besoin exprimé. Disposer d'un partenaire d'entraînement à l'oral français qui corrige de façon constante, ne se fatigue pas, reste disponible à toute heure, et dont le comportement de coaching est entièrement paramétrable. Trois usages sont visés, par ordre de fréquence attendue : la **préparation de soutenance**, la **préparation d'entretiens d'embauche**, et l'**entretien de l'expression professionnelle au quotidien**. Volume de référence : 10 à 20 minutes par jour.

Analyse d'écart avec les assistants vocaux du marché. Quatre points bloquants : la confidentialité (des enregistrements vocaux personnels partent chez un tiers), le coût récurrent, la latence dépendante du réseau, et surtout l'absence totale de contrôle sur le comportement de correction — c'est précisément la variable que le produit doit maîtriser.

Hierarchisation des besoins (matrice impact / effort). J'ai arbitré les fonctions candidates avant d'écrire la moindre ligne de code, et cet arbitrage est resté l'instrument de pilotage du sprint :

Besoin / fonction	Impact	Effort	Décision
Comportement de coaching maîtrisé et mesurable	fort	moyen	P0 — cœur du projet
Éveil vocal fiable sur l'appareil (confidentialité)	fort	moyen	P0 — cœur du projet
Fin de tour maîtrisée par l'apprenant	fort	fort	P1 — traité en fin de sprint
Mise à jour à distance du terminal (BLE)	faible	fort	réduit à une conception documentée
Tableau de bord de dérive	moyen	fort	réduit à des métriques journalisées
Extension à l'assistant domestique	faible	moyen	abandonné

Objectifs techniques visés. Le persona « coach » sert de banc d'essai à deux pratiques d'ingénierie IA, cœur méthodologique du projet :

1. **Cycle de vie complet d'un modèle de détection de mot d'éveil (KWS)** : corpus, entraînement, quantification INT8 (réduction des poids à des entiers 8 bits pour tenir sur le microcontrôleur), déploiement embarqué, mesure terrain, diagnostic, ré-entraînement, extension à un second modèle.
2. **Comportement du coach : prompt, puis fine-tuning, mesurés** — d'abord implémenté par instructions système, itéré jusqu'à un plafond mesuré, puis appris dans les poids, et comparé sur des **jeux tenus à l'écart** (jamais vus à l'entraînement).

Parties prenantes. Utilisatrice finale : l'apprenante, sur les trois usages ci-dessus. Commanditaire : Charlotte (FalDatacorps), dont le critère d'acceptation est la démontrabilité des compétences. Exploitant : moi-même, ce qui fait de la maintenabilité un critère de conception. Contrainte transverse : l'assistant vocal du foyer ne doit jamais être dégradé par le projet.

Contraintes réglementaires et éthiques. Les données vocales sont des données personnelles sensibles : aucune donnée audio ni texte ne quitte le réseau domestique, y compris pour la correction grammaticale. La minimisation est structurelle et non déclarative : rien n'est transmis tant que le mot d'éveil n'a pas été détecté sur le terminal.

2. Audit de la solution data existante (ou proposée)

2.1 Solution actuelle ou proposée

La brique « cerveau » réutilise une chaîne vocale auto-hébergée **déjà en production** et vérifiée :

- **Transcription** : Wyoming Whisper (`faster-whisper large-v3`, français).
- **Modèle de dialogue** : Ollama `qwen2.5:14b` avec un persona coach dédié.
- **Synthèse** : Wyoming Piper, voix `fr_FR-siwis-medium`.

- **Orchestration** : pipeline Home Assistant Assist **additif** — l'assistant domestique existant reste intact, le coach n'est jamais l'assistant par défaut.
- **Sécurité** : pare-feu hôte en liste blanche (seule la machine d'orchestration atteint les ports vocaux), valeurs réseau réelles hors des fichiers versionnés (le dépôt du projet est public).

2.2 Évaluation de l'adéquation aux besoins

Critère	Cloud (rejeté)	Auto-hébergé (retenu)
Confidentialité	données vocales chez un tiers	100 % local
Coût	récurrent (chiffré au §4.2)	matériel amorti, coût marginal nul
Latence	dépend du réseau	réseau domestique, maîtrisée
Contrôle du comportement	limité aux paramètres exposés	total (prompt et poids du modèle)
Observabilité	boîte noire	métriques et journaux maîtrisés

Écart matériel audité — point de méthode. Une hypothèse initiale du projet envisageait un « LLM hors ligne directement sur l'ESP32 ». J'ai réfuté cette hypothèse par une **arithmétique mémoire** : 8 Mo de PSRAM face à environ 1,6 Go de poids de modèle, soit deux ordres de grandeur d'écart. C'est cet audit qui a motivé l'architecture en cascade décrite au §3 : un petit modèle toujours actif au bord ouvre l'accès à un grand modèle à la demande sur le réseau local.

2.3 Limites auditées de la brique d'orchestration (sous-section ajoutée)

Un second audit, mené sur un défaut d'usage réel (le coach coupe l'apprenante quand elle réfléchit), illustre la même discipline : remonter à la source plutôt que régler des paramètres à l'aveugle. En lisant le code du serveur d'orchestration, j'ai établi que :

- la fin de tour est décidée **côté serveur** — le réglage embarqué « détection de silence » est purement et simplement ignoré — avec un seuil le plus permissif à 1,25 s et une **limite dure de 15 s** de capture, non exposée dans l'interface ;
- l'action d'arrêt du terminal **annule** le traitement (aucune transcription, aucune réponse) au lieu de le clôturer, alors que le protocole contient un signal de clôture gracieuse, simplement non câblé.

Conclusion : « laisser l'apprenante marquer une pause aussi longue qu'elle le souhaite » n'est pas un réglage, c'est une modification d'architecture — spécifiée et développée (§6). Le compromis « bouton pousser-pour-parler » a été écarté par choix de conception : l'expérience doit rester 100 % vocale.

3. Identification d'une solution technique cible

Méthodologie d'identification et de priorisation des cas d'usage. J'ai dérivé quatre cas d'usage du besoin (correction en direct, simulation de soutenance, simulation d'entretien, conversation libre), puis je les ai priorisés selon le rapport valeur pédagogique / effort. La simulation de soutenance est retenue comme cas d'usage de référence : c'est elle qui structure le jeu d'évaluation et le corpus d'entraînement, les autres cas partageant le

même comportement de coach sans développement spécifique. L'échelle de réduction de périmètre, écrite en semaine 1, fixait par avance l'ordre d'abandon en cas de dérapage — arbitrer sans renégocier l'objectif.

Architecture cible : cascade bord → cerveau.

Terminal ESP32-S3		Station GPU (RTX 5080)	
+-----+		+-----+	
Detection mot d'eveil	eveil >	Orchestration Assist	
Front-end audio	audio >	Transcription Whisper	
Micro I2S / HP / OLED	< synthese	Coach (modele de dialogue)	
+-----+		Synthese Piper (FR)	
		+-----+	

Justification des choix :

- **Bord** : ESP32-S3 N16R8, micro I2S INMP441, amplificateur MAX98357, écran OLED d'état. Le modèle de mot d'éveil tourne sur la carte ; rien ne transite tant qu'il ne s'est pas déclenché (confidentialité et sobriété réseau).
- **Modèle d'éveil** : « hello lingorm » entraîné sur mesure, quantifié **INT8 streaming**, déployé avec un seuil de coupure calibré sur un balayage de jeu tenu à l'écart (0,6 après ré-entraînement).
- **Cerveau** : modèle de dialogue de 14 milliards de paramètres en persona coach, transcription et synthèse françaises, orchestrés en réseau local.
- **Sécurité** : liaison locale uniquement, pare-feu hôte en liste blanche, secrets hors dépôt.

Comparatif des leviers de comportement (chiffres mesurés détaillés au §5) :

Levier	Ce qu'il apporte	Ce qu'il coûte	Constat mesuré
Prompt système	rapide, réversible, zéro entraînement	plafonne	plancher de conformité relevé, puis plafond atteint sur la tolérance aux hésitations
Fine-tuning (adaptateur QLoRA : petites matrices ajoutées au modèle gelé, entraînées en 4 bits)	comportement appris, stable	corpus à construire, passe GPU	franchit le plafond, au prix d'un rappel plus faible
Ancrage grammatical déterministe	exactitude factuelle	couverture limitée	corrige la fiabilité des règles, sans regagner le rappel

Le principe directeur retenu : **le fine-tuning façonne le comportement, pas le savoir ; l'ancrage déterministe façonne les faits.**

Statut de mise en service, dit explicitement. La chaîne vocale qui tourne aujourd'hui à la maison utilise le modèle de dialogue standard avec le prompt v3. Le fine-tune est **validé hors ligne** : le moteur d'inférence en service ne sait pas charger l'adaptateur, et sa mise en production suppose un moteur capable de le servir — étape identifiée, non réalisée dans cette itération.

4. Stratégie de mise en œuvre et d'industrialisation

4.1 Proposition de démarche projet

Sprint de 4 semaines, trois points de mentorat, découpé en vagues :

- **S1** — cadrage, inventaire des compétences (carte mentale), infrastructure et chaîne vocale.
- **S2** — cœur méthodologique : harnais d'évaluation et mesure de la baseline *avant* toute optimisation, modèle d'éveil, corpus de fine-tuning.
- **S3** — mise en service matérielle du terminal, cycle de vie du modèle d'éveil, itérations de prompt mesurées, entraînement du fine-tune.
- **S4** — portfolio, rapport, soutenance.

Gouvernance et outillage :

- Suivi par tickets (référence interne CS-145 à CS-159), un ticket par lot livrable, avec critères d'acceptation écrits avant le développement.
- Git Flow : branches de fonctionnalité, intégration sur `develop`, jalons taggés.
- **Porte qualité obligatoire par commit** : aucun commit n'est intégré sans une revue de code dédiée **et** une passe de tests verte.
- Rapports d'exécution archivés à chaque lot (revue, tests, résultats de mesure), ce qui rend chaque chiffre du §5 traçable jusqu'à son exécution.

Réduction de périmètre maîtrisée. J'ai décidé et tracé deux réductions plutôt que de les subir : la mise à jour à distance par BLE est documentée comme **conception** (architecture et diagrammes de séquence) et non implémentée — le flash reste manuel sur cette itération ; le tableau de bord de dérive est ramené à des métriques journalisées et analysées.

4.2 Aide à la prise de décision

Risques identifiés et traitements :

Risque	Traitement	Statut
Saturation mémoire GPU (16 Go partagés avec la chaîne vocale du foyer)	fenêtre d'exclusivité GPU libérant ~4 Go et déchargement préalable du modèle de dialogue résident (~10 Go), allocation mémoire défragmentée, redémarrage automatique des services à la sortie	traité ; le premier entraînement avait échoué faute d'avoir recensé les deux consommateurs
Rappel du mot d'éveil insuffisant sur voix réelle	mesure terrain, puis ré-entraînement avec voix réelle et bruit de fond	traité (§5)
Le prompt ne suffit pas à obtenir le comportement visé	mesure du plafond, puis fine-tuning justifié par la mesure	traité (§5)
Modèle qui fabrique des règles de grammaire	explication générée par gabarits déterministes + filtre de précision	traité, au prix du rappel

Risque	Traitement	Statut
Liaison Wi-Fi instable du terminal	diagnostic mené jusqu'à la cause environnementale (brouillage radio), auto-reconnexion périodique du terminal	contourné, correctif physique identifié

Indicateurs de succès retenus : conformité du format de correction (D1), persistance du français sous pression de changement de langue (D2), robustesse en contexte long (D3), respect des signaux de clôture (D4), tolérance aux hésitations et aux tours incomplets (D5), rappel des vraies erreurs (part des vraies erreurs effectivement corrigées), et côté embarqué le taux de réveil sur voix réelle.

Décision d'architecture économique : GPU local possédé contre services cloud. La comparaison qui a réellement gouverné la décision oppose la station GPU déjà en service à une reconstitution du même pipeline en services cloud.

Poste (usage de 10 à 20 min/jour, 30 j/mois)	Auto-hébergé (GPU déjà possédé)	Services cloud équivalents
Investissement initial	0 € (station déjà en service, mutualisée)	0 €
Terminal : carte ESP32-S3, micro INMP441, amplificateur MAX98357, écran, haut-parleur	25 à 40 € (<i>estimation, pas de facture disponible</i>)	25 à 40 € (identique)
Transcription + dialogue + synthèse	électricité seulement (quelques euros par mois)	2 à 18 €/mois hors paliers gratuits
Données vocales	ne quittent jamais le réseau domestique	transitent chez un tiers
Contrôle du comportement	total	limité aux paramètres exposés

J'ai chiffré la colonne cloud à partir des grilles tarifaires publiques des éditeurs (OpenAI, Google Cloud, Microsoft Azure, Deepgram, ElevenLabs), relevées le 22 juillet 2026 et converties au taux de référence de la Banque centrale européenne du même jour (1 € = 1,1408 USD) : pour 5 à 10 heures de transcription, 900 à 1 800 tours de dialogue et 150 000 à 300 000 caractères synthétisés par mois, l'exploitation ressort **de l'ordre de 2 à 18 € par mois** selon que l'on retienne des briques d'entrée de gamme ou des briques temps réel avec une voix haut de gamme. Deux enseignements :

- **Le poste dominant n'est pas le modèle de langage** — moins de 12 % du total — mais les deux conversions audio, à parts comparables. Contre-intuitif : dans un pipeline vocal, ce n'est pas l'intelligence qui coûte cher, c'est l'audio aux deux extrémités.
- **Les paliers gratuits mensuels couvrent l'intégralité du volume bas et la totalité de la synthèse dans les deux scénarios** (5 h de transcription et 0,5 M de caractères chez Azure, 4 M de caractères chez Google) ; au-delà de 5 h de transcription par mois, le poste de reconnaissance vocale devient payant. La fourchette annoncée est un coût de régime permanent une fois ces paliers dépassés, pas une dépense constatée.

Lecture honnête. Ce chiffrage ne plaide pas en faveur de l'auto-hébergement : à ce volume, le cloud aurait coûté zéro pendant longtemps. **L'argument décisif n'est donc pas le coût, mais la confidentialité des données vocales, la latence maîtrisée et le**

contrôle total du comportement — les trois critères posés au §1. Le coût ne redevient un argument qu'à volume multiplié ou en usage multi-postes. Pour un lecteur partant de zéro, l'investissement de la station serait de l'ordre de 2 300 € et son amortissement face à un abonnement cloud se compterait en années : raison supplémentaire de fonder la décision sur la maîtrise des données plutôt que sur le budget.

Impacts (légaux, biais, sécurité). Confidentialité par conception ; biais possibles du correcteur grammatical et du modèle sur certaines tournures, atténués par un filtre de précision qui ne corrige que les catégories d'erreurs classifiables ; surface d'exposition réseau réduite à une liste blanche.

5. Contrôle et suivi du projet

5.1 Tableau de bord de pilotage

Méthodologie de gestion. J'ai piloté le sprint en **Kanban sur tickets**, avec un cycle de vie explicite (Backlog → prêt → en cours → tests → revue → terminé) et un découpage en vagues hebdomadaires servant de jalons. Les pratiques d'inspiration DevOps sont portées par la porte qualité par commit (revue + tests verts avant intégration) et par l'archivage systématique des rapports d'exécution.

Axe	Instrument de suivi
Délais	tickets par vague (CS-145 à CS-159), trois jalons de mentorat, échéance de soutenance fixe
Coûts	GPU local, aucun coût de service externe ; suivi du temps par lot
Livrables	harnais d'évaluation, modèles d'éveil, fine-tune, couche d'ancrage, firmware, carte mentale, portfolio, rapport
Qualité	revue de code + passe de tests obligatoires par commit ; 264 tests unitaires et d'intégration
Performances	métriques déterministes sur jeux gelés + relecture qualitative indépendante

Résultat 1 — cycle de vie du modèle d'éveil (mesure, diagnostic, ré-entraînement).

Entraîné sur des voix synthétiques et évalué sur de l'audio propre, le premier modèle semblait excellent en laboratoire. Mesuré sur le terrain, avec le micro et ma voix réels : **1 réveil sur 8**. Diagnostic : ni voix réelle dans le corpus, ni bruit de fond dans l'augmentation de données. Après ré-entraînement (58 clips réels, augmentation par bruits d'environnement, ré-extraction des caractéristiques), le taux de réveil terrain passe à **8 sur 10**. Le taux de faux rejet est *monté* de ~ 0 à $\sim 0,095$: ce n'est pas une régression, c'est le jeu de test qui a cessé d'être optimiste.

J'ai ensuite rejoué le cycle complet sur un **second modèle**, le mot de fin de tour « j'ai fini » (99 clips réels + 4 961 synthétiques) : entraîné, quantifié INT8 streaming, balayage de seuil donnant un taux de faux rejet de **0,0197 au seuil 0,49 avec zéro faux déclenchement par heure**. Ce dernier chiffre est **optimiste pour le français** — les négatifs d'entraînement sont anglophones alors que « j'ai fini » est une locution française courante — d'où un seuil déployé volontairement conservateur (0,6) et un étalonnage sur l'appareil requis avant mise en service.

Résultat 2 — itérations de prompt, jusqu'au plafond mesuré. J'ai spécifié, déployé et mesuré deux itérations successives :

Dimension	prompt v1	prompt v2	prompt v3
D4 — reconnaissance d'un signal de clôture (7 cas)	0/7	7/7	— (non re-mesuré) *
D4 — clôtures prématurées	0 %	0 %	— (non re-mesuré) *
D5 — fausses corrections sur hésitation (8 cas, plus bas = mieux)	—	0,50	0,50
D5 — invitation à poursuivre sur tour incomplet	—	0,00	0,25

* La v3 reprend le texte de la v2 mot pour mot et y ajoute deux paragraphes ; D4 n'a pas été ré-exécuté sur la v3.

La v2 règle un vrai défaut d'usage. La v3, ciblée sur la tolérance aux hésitations, **ne bouge pas la métrique** : 0,50 avant, 0,50 après. Une formulation plus insistante l'a même **dégradée** (0,625 contre 0,500) : insister sur la correction en augmente la saillance et pousse le modèle vers ce qu'on lui demandait d'ignorer. **C'est ce résultat négatif, mesuré et reproduit, qui justifie le fine-tuning** — et non une intuition.

Résultat 3 — fine-tuning, mesuré sur un jeu tenu à l'écart. Corpus porté de 80 à 125 exemples (dont 45 exemples « reconstruire puis corriger »), disjonction du corpus et des jeux d'évaluation prouvée par un contrôle de fuite automatisé. Entraînement d'un adaptateur QLoRA 4 bits sur le modèle de 14 milliards de paramètres, 4 époques / 56 pas, **8 min 11 s** sur GPU grand public (durée relevée dans le journal d'exécution archivé, `train_runtime` = 490,8 s), perte d'entraînement 0,552. Évaluation sur les 19 dialogues D5 tenus à l'écart, adaptateur activé puis désactivé sur des poids identiques :

Métrique D5 (19 dialogues tenus à l'écart)	baseline (sans adaptateur)	fine-tuné
Exactitude globale	0,579	0,895
Fausse corrections sur hésitation (plus bas = mieux)	0,500	0,125
Invitation à poursuivre sur tour incomplet	0,000	1,000
Rappel des vraies erreurs	1,000	0,750
Contrôle « tour propre non corrigé »	1,000	1,000

Mise en garde de taille d'échantillon, énoncée avant qu'on me la pose : D5 compte 19 items, un seul item déplace un taux de 0,125 à 0,25 ; tout écart inférieur à ~0,15 est du bruit ; et chaque bras n'a été exécuté qu'une fois en décodage déterministe, ce qui échantillonne un point du comportement, pas sa dispersion.

Deux enseignements de pilotage comptent plus que le gain lui-même :

- **Le coût est réel et il est publié.** Le rappel des vraies erreurs tombe de 1,00 à 0,75 : le modèle a échangé de la promptitude à corriger contre de la sûreté sur les hésitations. Le levier correctif est un rééquilibrage du corpus, pas davantage d'époques.

- **Prompt et poids ne se cumulent pas.** Les bras « adaptateur + prompt v3 » et « adaptateur + prompt v0 » donnent des métriques agrégées identiques : une fois le comportement dans les poids, le prompt v3 devient inutile — simplification directe du déploiement.

Résultat 4 — non-régression du fine-tune (jeu principal gelé, 18 dialogues exécutés, 2 répétitions) :

Métrique déterministe	baseline	fine-tuné (adaptateur v1)
D1 — conformité du format de correction	50,0 %	100,0 %
D1 — taux de faux positifs (plus bas = mieux)	50,0 %	0,0 %
D2 — persistance du français	55,6 %	88,9 % *

* Les deux seuls échecs D2 portent sur la même phrase française ordinaire, classée à tort comme non française par le détecteur de langue. Audit des 48 réponses : aucune n'est en anglais ni en chinois. La persistance réelle est de 100 % ; l'artefact de scoring est documenté et laissé tel quel pour ne pas modifier un jeu gelé en cours de comparaison.

Résultat 5 — apport de la couche d'ancrage grammatical. *Exécution distincte, menée sur l'adaptateur de référence de la session précédente : ces chiffres ne se lisent pas dans la continuité du tableau ci-dessus.*

Métrique	baseline	fine-tuné (adaptateur de référence)	+ ancrage
D1 — conformité du format	50,0 %	95,0 %	100,0 %
D1 — taux de faux positifs	50,0 %	12,5 %	12,5 % (inchangé)
D2 — persistance du français	55,6 %	100 %	100 %
Exactitude de l'explication grammaticale (relecture qualitative)	44,4 %	44,4 %	100 % sur les tours où la couche se déclenche

L'ancrage porte l'exactitude des explications de 44,4 % à 100 % là où il agit, sans dégrader la précision, mais il ne couvre qu'une partie des tours : la précision est achetée contre du rappel. C'est la preuve du principe « comportement par les poids, faits par l'ancrage ».

5.2 Outils et process de suivi

- **Jeux d'évaluation gelés** : un jeu principal de 18 dialogues exécutés (20 spécifiés) couvrant D1, D2 et D3, et **deux mini-jeux ciblés** (D4 clôture, 21 dialogues ; D5 hésitations, 19 dialogues). La couche d'ancrage n'a pas de jeu propre : elle est évaluée comme troisième bras sur le jeu principal. Les textes de l'apprenante sont verrouillés par un contrôle automatique qui échoue si le harnais dévie de la spécification.
- **Séparation entraînement / évaluation prouvée, pas supposée** : un contrôle de fuite compare le corpus à chaque jeu d'évaluation et échoue explicitement en l'absence de jeu à contrôler (garde contre le faux succès).
- **Méthodologie de test** : 264 tests unitaires et d'intégration exécutés à chaque commit (harnais, scoreurs, corpus, chaîne de fine-tuning, artefacts des modèles d'éveil, démonstrateur, intégration serveur), complétés par des bancs d'essai sur matériel réel

pour tout ce que la validation hors ligne ne prouve pas. Aucun test de charge : le service est mono-utilisateur.

- **Deux mesures indépendantes** : métriques déterministes du harnais et relecture qualitative des transcriptions, concordantes sur la session de référence. L'exécution du fine-tune final n'a pas fait l'objet d'une relecture, ses métriques D5 étant entièrement déterministes.
- **Portes de qualité des données** : depuis le second modèle d'éveil, chaque lot audio généré passe une porte d'intelligibilité automatique (transcription puis vérification de la phrase, seuil 90 %). Elle a évité une erreur coûteuse : un générateur produisait des clips de français fluide **mais sans rapport avec la phrase demandée** en dessous de six à sept mots. Sans cette vérification, le second modèle aurait été entraîné sur du bruit vraisemblable.
- **Traçabilité** : verdicts bruts, configurations d'exécution et empreintes des prompts archivés à chaque passe.

6. Conclusion et recommandations

Enseignement principal : mesurer d'abord, optimiser ensuite. J'ai construit le harnais d'évaluation et gelé les jeux *avant* toute optimisation du comportement. C'est ce qui m'a permis de démontrer qu'une itération de prompt plafonnait (0,50, inchangé, et dégradé quand on insiste), donc de justifier le fine-tuning par une mesure plutôt que par une intuition — puis de publier ce qu'il a coûté (rappel 1,00 → 0,75) au lieu de ne montrer que le gain.

Enseignements secondaires (détail et chiffres au §5.1) :

- Un modèle validé en laboratoire peut échouer sur le terrain : le cycle de vie ne se termine pas au déploiement, il y commence. Et une métrique qui se dégrade peut signifier que le test est devenu honnête, pas que le modèle a régressé.
- Le fine-tuning façonne le comportement, pas le savoir : l'exactitude des explications grammaticales reste à 44,4 % avec ou sans adaptateur, et seule une couche déterministe la porte à 100 %, au prix du rappel.
- En débogage système, la source fait foi : j'ai établi la propriété de la fin de tour en lisant le code du serveur, après deux hypothèses plausibles et fausses.
- Les portes de qualité des données valent leur coût : elles ont intercepté un générateur produisant du français fluide mais faux.

Mise en service réalisée (soir du 22/07). La clôture de tour par mot de fin a été déployée et **validée en direct sur l'appareil** : brique d'orchestration (Home Assistant) mise à niveau (conteneur précédent conservé pour retour arrière, sauvegarde de configuration prise avant migration), entité de transcription installée et sélectionnée dans le pipeline, firmware flashé. Résultats mesurés (extraits horodatés du journal série consignés dans `HW_VALIDATION_CS159.md`) : tours de 21 à 35 secondes avec pauses libres (la coupe à 1,25 s et le plafond de 15 s ont bien disparu) ; 7 détections du mot de fin sur 7 pendant la capture active ; la clôture gracieuse livre transcript et réponse dès que le tour contient de la parole. Un défaut produit a été découvert en conditions réelles et corrigé dans la soirée : le mot de fin prononcé figurait dans le transcript et était lu comme un adieu par les règles de clôture du coach ; correction déterministe dans l'entité de transcription (suppression des

occurrences finales uniquement, usages en milieu de phrase préservés), couverte par des tests dédiés et revalidée à l'oreille. Un défaut d'ergonomie reste ouvert : dans les secondes qui suivent la fin d'un tour, le premier réveil peut être avalé (répéter le mot d'éveil suffit ; filet de sécurité réarmé en 1 s).

Durcissement et revalidation (24/07). L'usage réel a ensuite révélé une variante de mistranscription du mot de fin (« Réfinis ») qui échappait au retrait ; la correction a été étendue à cette variante observée et durcie au passage — restructuration du motif contre un emballement combinatoire détecté par la porte de test (l'entrée répétée observée en production aurait gelé la boucle d'événements de l'orchestrateur), normalisation Unicode du transcript, et biais de vocabulaire ajouté à la transcription. La suite de tests de l'entité compte désormais 80 cas (dont 48 dédiés au retrait du mot de fin et 3 tests de non-régression de performance). Redéployée puis **revalidée en direct le 24/07** : 4 tours en voix réelle, y compris des répétitions du mot de fin en rafale, tous correctement clôturés sans fausse lecture d'adieu.

Prochaines étapes recommandées, par ordre de valeur décroissante :

1. **Corriger le réveil avalé juste après un tour** (défaut R1 ci-dessus) : trace source de la course entre la fin du pipeline côté serveur et la nouvelle demande de démarrage, puis correctif.
2. **Étalonner sur l'appareil le seuil du second modèle d'éveil** (0,6 déployé) : les faux déclenchements sont sous-estimés par des négatifs anglophones.
3. **Rééquilibrer le corpus de fine-tuning** pour regagner le rappel perdu (davantage d'exemples de vraie erreur dans un énoncé hésitant).
4. **Livrables OpenClassrooms : publier la branche `develop` sur le miroir public — prérequis bloquant de la remise** —, publier le portfolio, exporter le présent rapport, préparer la soutenance et les questions/réponses.

7. Annexes

Dépôt et documentation

- Dépôt du projet : <https://github.com/Lesliam/openclassrooms-project15> (branche `develop` publiée sur le miroir public).
- README du projet : architecture, décisions de conception, topologie.

Annexe A — Évaluation et mesures

- `ml/eval/eval_set_v0.md` — jeu gelé (D1, D2, D3) et protocole.
- `ml/eval/closing_cue_set_v1.md` + `CLOSING_CUE_REPORT.md` — dimension D4.
- `ml/eval/disfluency_set_v1.md` + `DISFLUENCY_REPORT.md` — dimension D5, plafond du prompt ; y figure l'essai exploratoire de formulation plus insistante (0,875), non archivé et signalé comme tel.
- `ml/eval/harness/` — harnais, scoreurs déterministes, contrôle de conformité.

Annexe B — Fine-tuning

- `ml/finetune/RESULTS_CS158.md` — tableaux complets prompt vs fine-tuné, échecs résiduels et limites de l'étude.
- `ml/finetune/DATA_SEPARATION.md` + `check_leakage.py` — preuve de disjonction corpus / jeux d'évaluation.
- `ml/finetune/sft_config.yaml`, `train_sft.py` — configuration et lanceur.
- `ml/finetune/results/cs158/logs_train_cs158.txt` — journal d'exécution de l'entraînement (`train_runtime` 490,8 s, perte 0,5521), source de la durée citée au §5.1.

Annexe C — Cycle de vie des modèles embarqués

- `ml/kws/RETRAIN_WITH_REAL_VOICE.md` — diagnostic du 1/8 et procédure de ré-entraînement voix réelle + bruit de fond.
- `ml/kws/models/` — les deux modèles INT8 déployés et leurs manifestes (« hello lingorm », « j'ai fini »).
- `ml/kws/models/jai_fini_tflite_streaming_roc.txt` — balayage de seuil du second modèle (faux rejet 0,0197 au seuil 0,49, zéro faux déclenchement par heure).
- `ml/kws/PREP_JAI_FINI.md` — document de préparation du second modèle (recette validée, porte d'intelligibilité, coût de la diversité de locuteurs).

Annexe D — Terminal et système

- `firmware/esphome/` — configurations du terminal (mise en service par étapes, interface d'état, production, intégration du mot de fin).
- `firmware/esphome/VAD_OPTIONS_CS159.md` — audit de la fin de tour, options comparées, recommandation ; `BENCH_RESULT_CS159.md` — banc d'essai matériel.
- `firmware/esphome/HW_VALIDATION_CS159.md` — validation matérielle en direct de la clôture par mot de fin (22/07 soir, complétée le 24/07) : résultats par test, défauts trouvés et corrigés, défaut résiduel documenté.
- `firmware/docs/OTA_BLE_DESIGN.md` — **conception de la mise à jour à distance par BLE** (fonction réduite à une conception documentée, cf. §4.1) : transport par canal L2CAP orienté connexion plutôt que GATT pour le débit, sécurité en mode 1 niveau 4 avec amorçage par matériel pré-provisionné en usine, plan de robustesse et estimation d'effort. La voie de mise à jour Wi-Fi existante, suffisante pour le MVP, a été **vérifiée en fonctionnement le 24/07** (téléversement de 1,28 Mo en 4,9 s, redémarrage et retour en ligne automatiques).
- Fiche de câblage du terminal (brochage vérifié sur la carte).

Annexe E — Éléments budgétaires et livrables du portfolio

- `deliverables/rapport/CLOUD_PRICING_EVIDENCE.md` — dossier de preuves tarifaires du §4.2 : relevés éditeur par éditeur avec URL et date, hypothèses d'usage, calcul des deux scénarios, effet des paliers gratuits, limites du chiffrage.
- Carte mentale des compétences ; démonstrateur web du coach ; diagrammes d'architecture et conception de la mise à jour à distance.
- Visuels à joindre : schéma de l'architecture en cascade, capture du démonstrateur, photographie du terminal en fonctionnement, graphique baseline / fine-tuné sur D5.